

CISC, RISC, VLIW and Superscalar Architectures

RISC and CISC



What is CISC?

- CISC is an acronym for Complex Instruction Set Computer and are chips that are easy to program and which make efficient use of memory. Since the earliest machines were programmed in assembly language and memory was slow and expensive, the CISC philosophy made sense
- Most common microprocessor designs such as the Intel 80x86 and Motorola 68K series followed the CISC philosophy.
- But recent changes in software and hardware technology have forced a re-examination of CISC and many modern CISC processors are hybrids, implementing many RISC principles.
- CISC was developed to make compiler development simpler. It shifts most of the burden of generating machine instructions to the processor. For example, instead of having to make a compiler write long machine instructions to calculate a square-root, a CISC processor would have a built-in ability to do this.

Most CISC hardware architectures have several characteristics in common:

- Complex instruction-decoding logic, driven by the need for a single instruction to support multiple addressing modes.
- A small number of general purpose registers. This is the direct result of having instructions which can operate directly on memory and the limited amount of chip space not dedicated to instruction decoding, execution, and microcode storage.
- Several special purpose registers. Many CISC designs set aside special registers for the stack pointer, interrupt handling, and so on. This can simplify the hardware design somewhat, at the expense of making the instruction set more complex.
- A 'Condition code' register which is set as a side-effect of most instructions. This register reflects whether the result of the last operation is less than, equal to, or greater than zero and records if certain error conditions occur.

CISC Summary

CISC (Complex Instruction Set Computing)

- instructions are quite complex and have variable length.
- a relatively small number of registers, and are capable of accessing memory locations directly.
- Complex instructions are sequenced in microcode in modern CISC processors.

What is RISC?

- **RISC?**

RISC, or *Reduced Instruction Set Computer*, is a type of microprocessor architecture that utilizes a small, highly-optimized set of instructions, rather than a more specialized set of instructions often found in other types of architectures.

- **History**

The first RISC projects came from IBM, Stanford, and UC-Berkeley in the late 70s and early 80s. The IBM 801, Stanford MIPS, and Berkeley RISC 1 and 2 were all designed with a similar philosophy which has become known as RISC. Certain design features have been characteristic of most RISC processors:

- *one cycle execution time*: RISC processors have a CPI (clock per instruction) of one cycle. This is due to the optimization of each instruction on the CPU and a technique called PIPELINING
- *pipelining*: a technique that allows for simultaneous execution of parts, or stages, of instructions to more efficiently process instructions;
- *large number of registers*: the RISC design philosophy generally incorporates a larger number of registers to prevent in large amounts of interactions with memory

RISC Attributes

The main characteristics of CISC microprocessors are:

- Extensive instructions.
- Complex and efficient machine instructions.
- Microencoding of the machine instructions.
- Extensive addressing capabilities for memory operations.
- Relatively few registers.

In comparison, RISC processors are more or less the opposite of the above:

- Reduced instruction set.
- Less complex, simple instructions.
- Hardwired control unit and machine instructions.
- Few addressing schemes for memory operands with only two basic instructions, LOAD and STORE
- Many symmetric registers which are organised into a register file.

RISC Summary

RISC(Reduced Instruction Set Computing)

- instructions are of fixed length and of a regular format.
- Operations are performed on registers only, of which a larger number is available than on CISC processors. The only memory operations are load and store.
- The hardware in RISC processors is simpler because the RISC architecture relies more on the compiler for sequencing complex operations.

The method for exploiting parallelism

- The key to higher performance in microprocessors for a broad range of applications is the ability to exploit fine-grain, instruction-level parallelism:
 - + pipelining
 - + multiple processors
 - + superscalar implementation
 - + specifying multiple independent operations per instruction

Problems we meet

- it is not easy to exploit parallel execution in real programs, which are written in a serial fashion.
- Mainstream high-level languages (C and FORTRAN) allow a limited freedom to execute operations in parallel.
- Programs need to be compiled into machine code, but most conventional instruction sets do not allow for the indication of parallel execution.

VLIW was invented



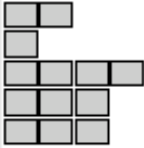
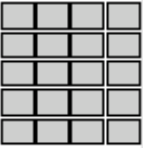
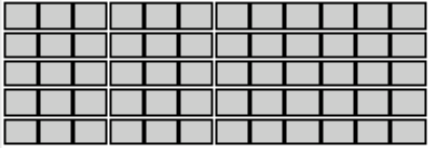
The idea of VLIW has been considered the work on trace scheduling, a method of compiling programs written in conventional languages for wide-word machines, done by Josh Fisher in 1979 at Yale laid down the foundation for VLIW technology. Now John Fisher leads HP's VLIW compiler project.

VLIW Pioneer: HP Senior Fellow Josh Fisher beside his MultiFlow Trace VLIW machine, on display at Computer History Museum.

Why VLIW ?

- To overcome the difficulty of finding parallelism in machine-level object code.
- In a VLIW processor, multiple instructions are packed together and issued in parallel to an equal number of execution units.
- The compiler (not the processor) checks that there are only independent instructions executed in parallel.

Comparison of VLIW, CISC, RISC

ARCHITECTURE CHARACTERISTIC	CISC	RISC	VLIW
INSTRUCTION SIZE	Varies	One size, usually 32 bits	One size
INSTRUCTION FORMAT	Field placement varies	Regular, consistent placement of fields	Regular, consistent placement of fields
INSTRUCTION SEMANTICS	Varies from simple to complex; possibly many dependent operations per instruction	Almost always one simple operation	Many simple, independent operations
REGISTERS	Few, sometimes special	Many, general-purpose	Many, general-purpose
MEMORY REFERENCES	Bundled with operations in many different types of instructions	Not bundled with operations, i.e., load/store architecture	Not bundled with operations, i.e., load/store architecture
HARDWARE DESIGN FOCUS	Exploit microcoded implementations	Exploit implementations with one pipeline and no microcode	Exploit implementations with multiple pipelines, no microcode & no complex dispatch logic
PICTURE OF FIVE TYPICAL INSTRUCTIONS  = 1 BYTE			

VLIW characteristics

- VLIW contains multiple primitive instructions that can be executed in parallel by functional units of a processor.
- The compiler packs a number of primitive, non-interdependent instructions into a very long instruction word
- Since multiple instructions are packed in one instruction word, the instruction words are much larger than CISC and RISC's.

The VLIW compiler

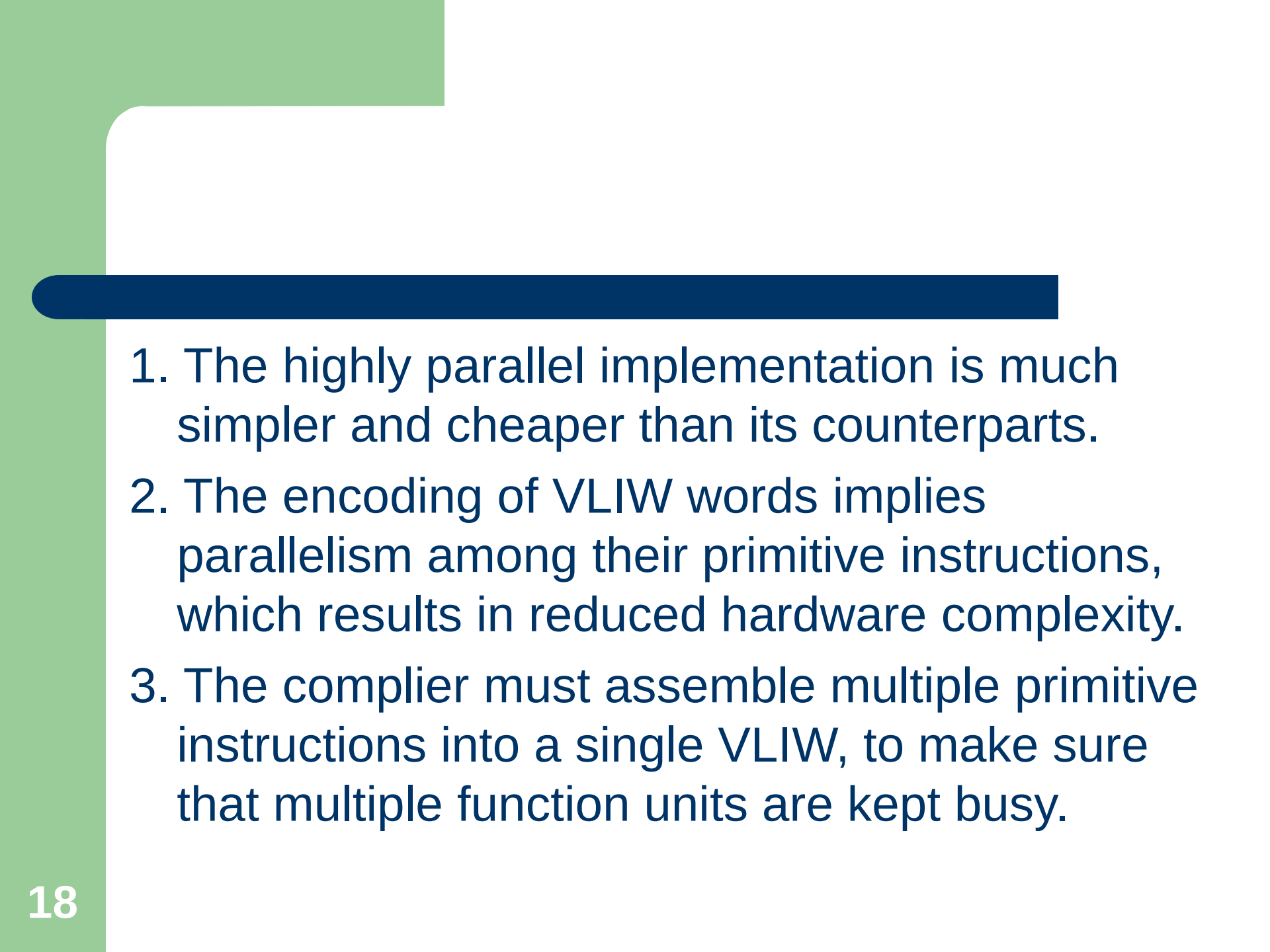
- The compiler specifies the primitive instructions per VLIW instruction word.
- The compiler must guarantee that the multiple primitive instructions which group together are independent so they can be executable in parallel.
- Only the sequence of different VLIW words affects the outputs (e.g., blue, red, green).

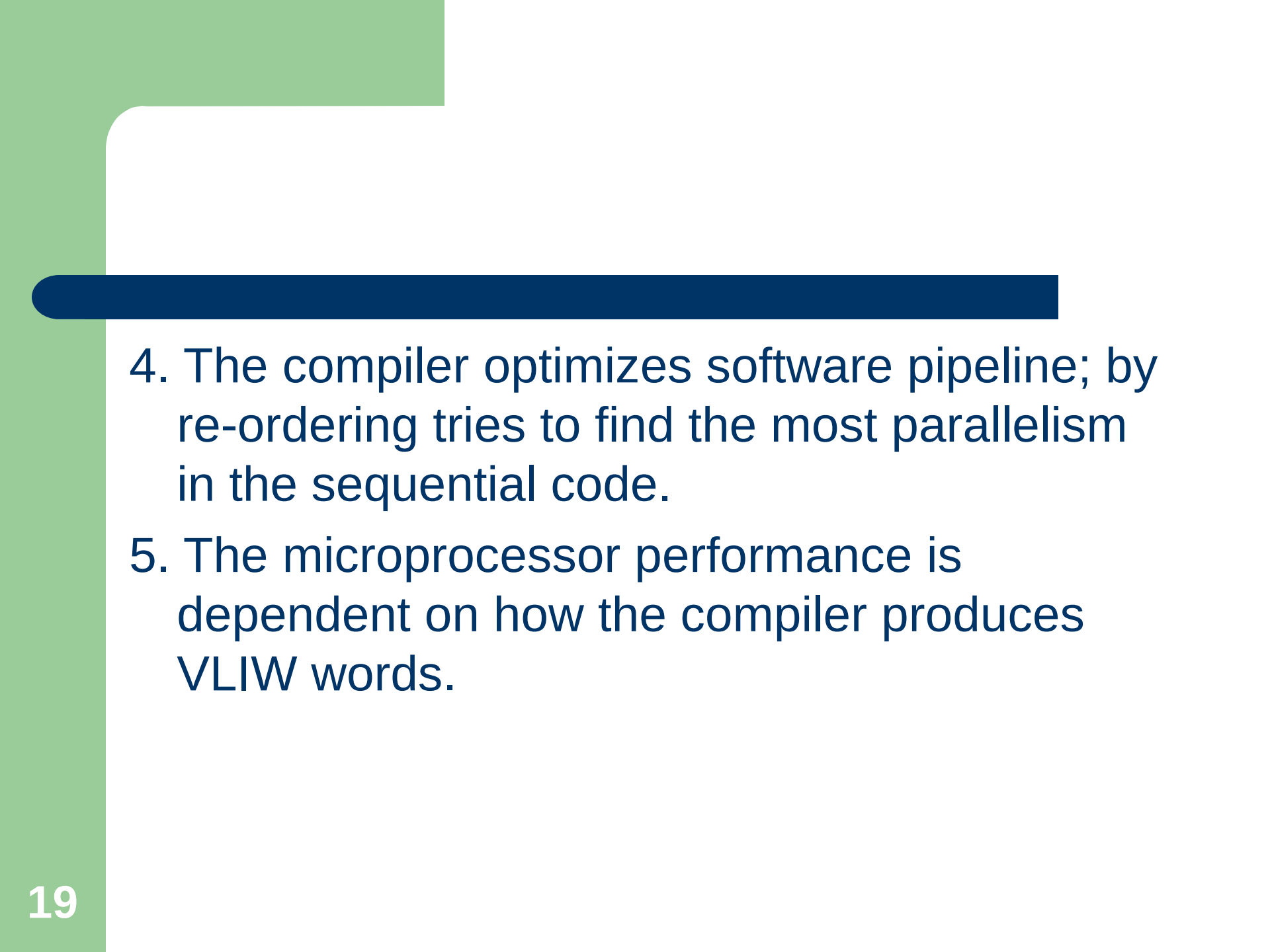
VLIW principles

1. The compiler analyzes dependence of all instructions among sequential code, tries to extract as much parallelism as possible.
2. Based on the analysis, the compiler re-codes the piece of sequential code in VLIW instruction words.
3. Finally, the work left with VLIW hardware is only fetch the VLIWs from cache, decode them, and then dispatch the independent primitive instructions to corresponding function units and execute.

SOFTWARE INSTEAD OF HARDWARE: IMPLEMENTATION ADVANTAGES OF VLIW

VLIW instructions explicitly specify several independent operations— decode the instruction and dispatch hardware that tries to reconstruct parallelism from a serial instruction stream. The processor does not need to consider whether or not the instructions are parallel.

- 
1. The highly parallel implementation is much simpler and cheaper than its counterparts.
 2. The encoding of VLIW words implies parallelism among their primitive instructions, which results in reduced hardware complexity.
 3. The compiler must assemble multiple primitive instructions into a single VLIW, to make sure that multiple function units are kept busy.

- 
4. The compiler optimizes software pipeline; by re-ordering tries to find the most parallelism in the sequential code.
 5. The microprocessor performance is dependent on how the compiler produces VLIW words.

Superscalar Architectures

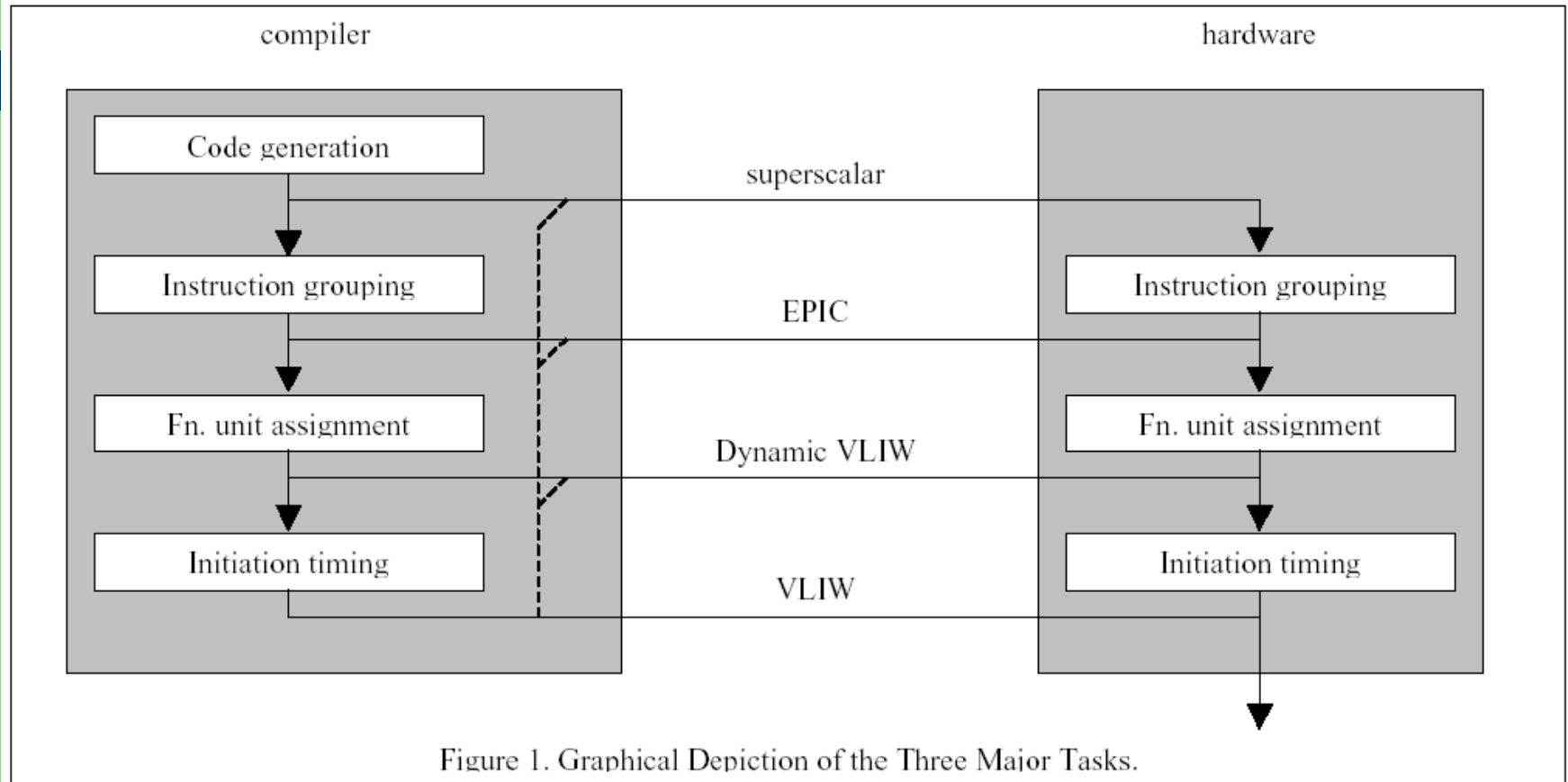
Major categories [1]

	Grouping	Fn unit asgn	Initiation
Superscalar	Hardware	Hardware	Hardware
EPIC	Compiler	Hardware	Hardware
Dynamic VLIW	Compiler	Compiler	Hardware
VLIW	Compiler	Compiler	Compiler

VLIW – Very Long Instruction Word

EPIC – Explicitly Parallel Instruction Computing

Major categories [2]



Superscalar Processors [1]

- Superscalar processors are designed to exploit more instruction-level parallelism in user programs.
- Only independent instructions can be executed in parallel without causing a wait state.
- The amount of instruction-level parallelism varies widely depending on the type of code being executed.

Pipelining in Superscalar Processors [1]

- In order to fully utilise a superscalar processor of degree m , m instructions must be executable in parallel. This situation may not be true in all clock cycles. In that case, some of the pipelines may be stalling in a wait state.
- In a superscalar processor, the simple operation latency should require only one cycle, as in the base scalar processor.

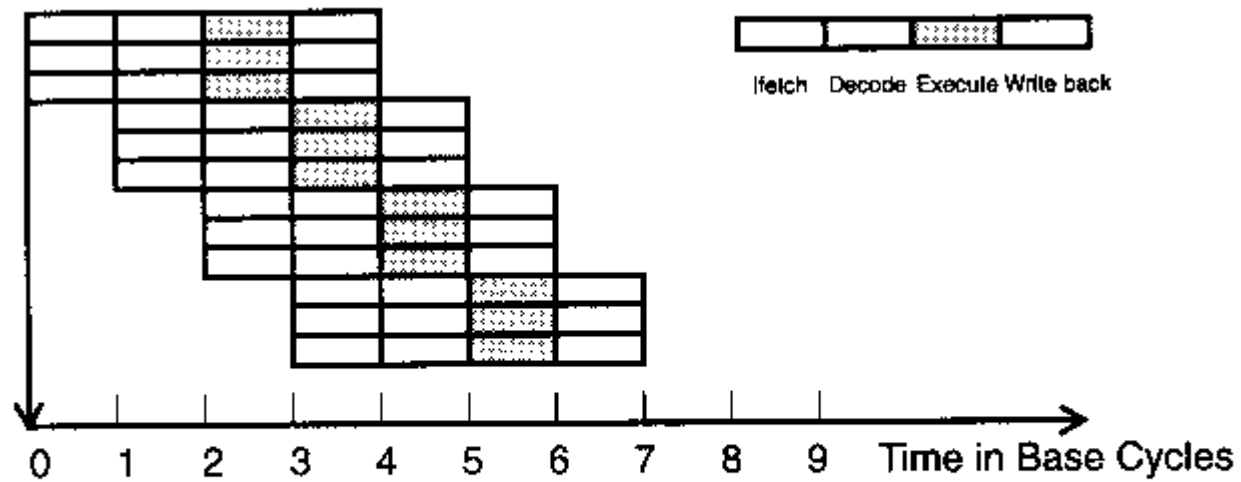
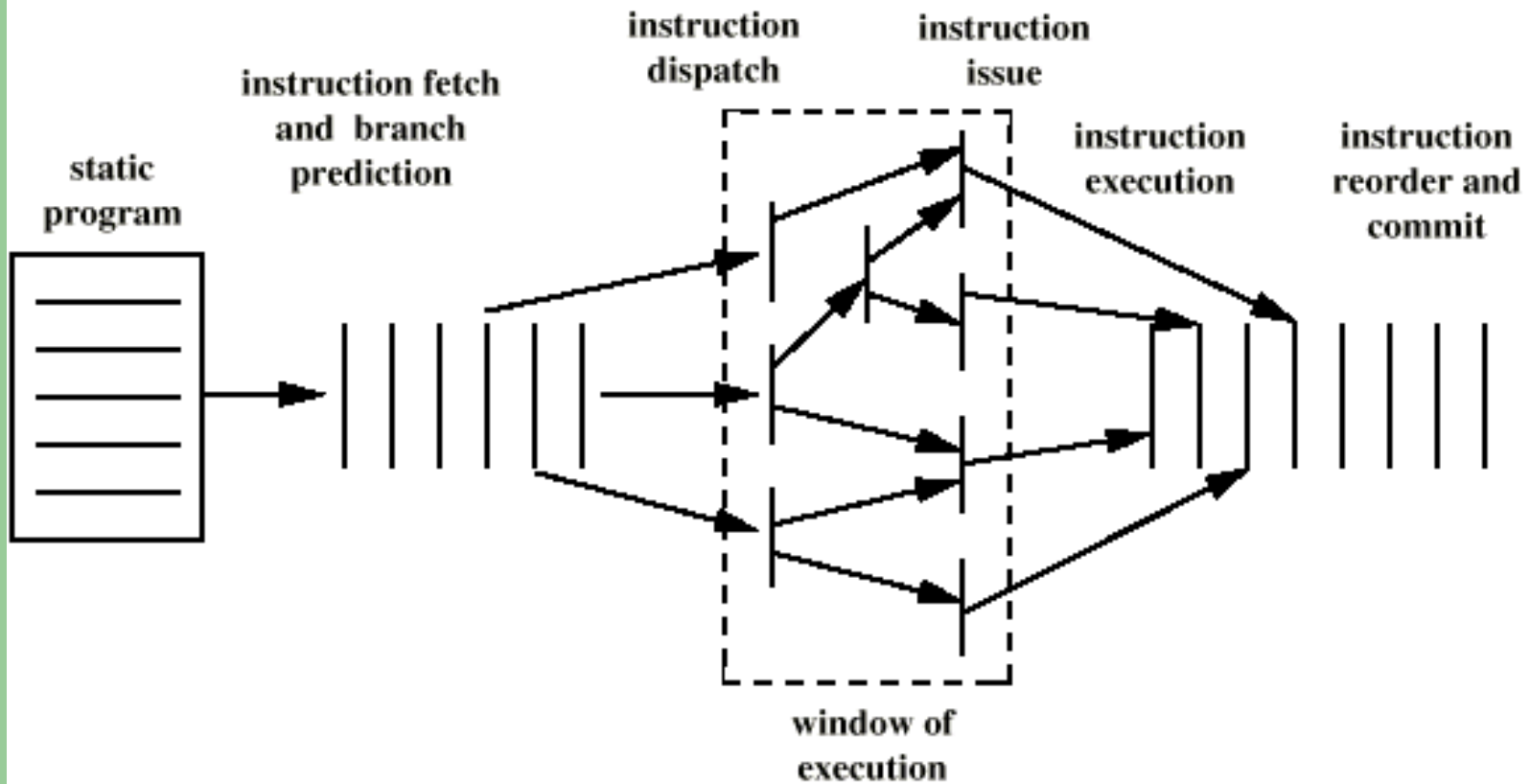


Figure 4.11 A superscalar processor of degree $m = 3$.

Superscalar Execution



Superscalar Implementation

- Simultaneously fetch multiple instructions
- Logic to determine true dependencies involving register values
- Mechanisms to communicate these values
- Mechanisms to initiate multiple instructions in parallel
- Resources for parallel execution of multiple instructions
- Mechanisms for committing process state in correct order